جامعة الأميرة
نورة بنت عبدالرحمن
كلية علوم الحاسب والمعلومات

Data Structures - CS 212t

Project

2024 – 1446

First Semester

A HOTEL BOOKING SYSTEM

O32

Student Name	Student ID	Student E-mail
لجين النتيفي	445009186	445009186@pnu.edu.sa
ليان فايز أحمد	445009155	445009155@pnu.edu.sa
شهد العتيبي	445009167	445009167@pnu.edu.sa
رغد القحطاني	445009195	445009195@pnu.edu.sa
غالية البخيت	445009172	445009172@pnu.edu.sa

`

I Project Introduction

1.1 Overview

Our project focuses on designing a Hotel Management System. The goal of this system is to simplify key hotel operations like managing guest information, booking rooms, and organizing client data efficiently. The system classifies rooms by type (e.g., single, double, or suites) and ensures streamlined management of guest data, making the process faster and more user-friendly.

1.2 Problem and solution

Hotels booking system face challenges like tracking guests, managing bookings, and assigning rooms. Keeping guest data up-to-date and handling bookings in real-time can be difficult, especially with many guests. To overcome these challenges, we have created methods that leverage advanced data structures to efficiently manage guest information, bookings, and room assignments.

1.3 Solution justification

We chose data structures that align with the system's operations:

1- Doubly Linked List: To store guest data, providing flexibility for efficient insertion, deletion, and updates.

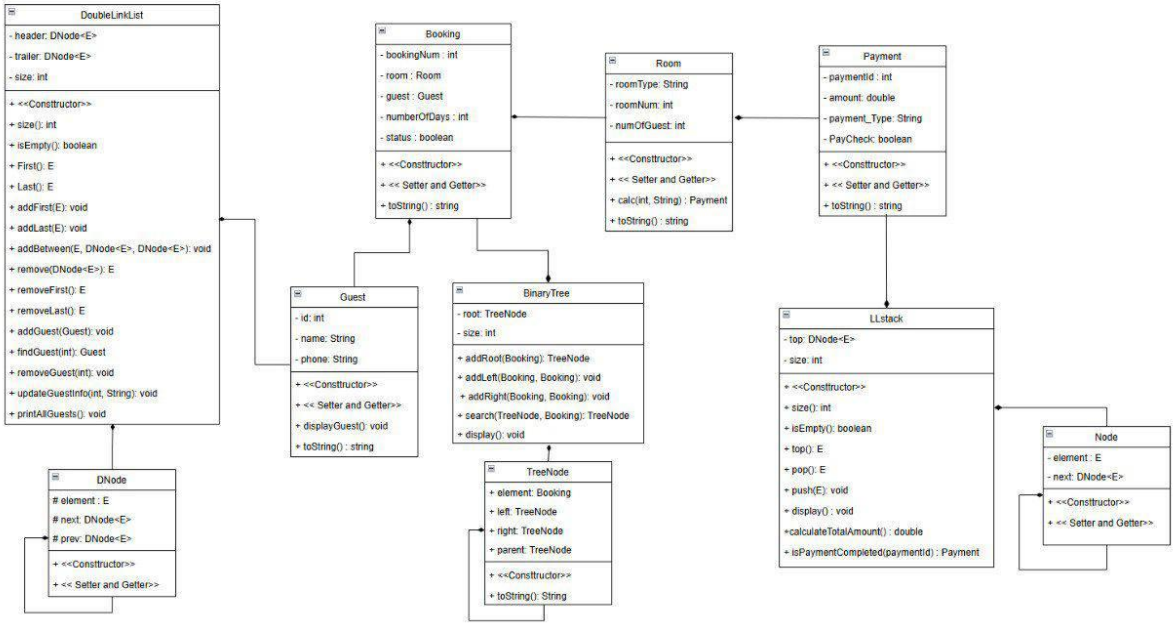
2- Binary Search Tree: To handle room data, ensuring easy sorting and searching.

3- Stack: To manage payment processes, allowing a structured and efficient way to handle transactions in a Last-In-First-Out (LIFO) manner.

These structures helped us create an efficient and well-organized system that effectively addresses the challenges of hotel management.

`

II Project class diagram



III Concepts covered

III.1.1 Doubly Linked List

III.1.2 Method #1 void addGuest(guest)

III.1.3 Method #2 Guest findGuest(int)

III.1.4 Method #3 void removeGuest(int)

III.1.5 Method #4 void updateGuestInfo(int,String)

III.1.6 Method #5 void printAllGuest()

`

```java
public Guest findGuest(int guestID) {
    DNode<E> current = header.getNext();
    while (current != trailer) {
        Guest guest = (Guest) current.getElement();
        if (guest.getId() == guestID) {
            return guest;
        }
        current = current.getNext();
    }
    return null;
}

public void removeGuest(int guestID) {
    DNode<E> current = header.getNext();
    while (current != trailer) {
        Guest guest = (Guest) current.getElement();
        if (guest.getId() == guestID) { // Corrected comparison
            current.getPrev().setNext( newNext: current.getNext());
            current.getNext().setPrev( newPrev: current.getPrev());
            size--;
            System.out.println("Guest " + guestID + " has been removed.");
            return;
        }
        current = current.getNext();
    }
    System.out.println( x: "Guest not found.");
}
```

```java
public void updateGuestInfo(int guestID, String newPhoneNumber) {
    Guest guest = findGuest(guestID);
    if (guest != null) {
        guest.setPhone( phone: newPhoneNumber);
        System.out.println( x: "Guest information updated successfully.");
    } else {
        System.out.println( x: "Guest not found.");
    }
}
```

`

III.2 Binary Search Tree

III.2.1 Method #1 TreeNode addRoot(Booking)
III.2.2 Method #2 void addLeft(Booking,Booking)
III.2.3 Method #3 void addRight(Booking,Booking)
III.2.4 Method #4 TreeNode search(Booking,Booking)
III.2.5 Method #5 void display()

```
private TreeNode<E> cancel(TreeNode<E> root, int bookingNum) {
    if (root == null) {
        System.out.println( x:"The Booking not found.");
        return null;
    }
    if (bookingNum < ((Booking) root.element).getBookingNum()) {
        root.left = cancel( root:root.left, bookingNum);
    } else if (bookingNum > ((Booking) root.element).getBookingNum()) {
        root.right = cancel( root:root.right, bookingNum);
    } else {
        if (root.left == null) {
            return root.right;
        }
        if (root.right == null) {
            return root.left;

        }
    }
    return root;
}

public void cancelBooking(int bookingNum) {
    Root = cancel( root:Root, bookingNum);
}

private void Display(TreeNode<E> node) {
    if (node != null) {
        Display( node:node.left);
        System.out.println(node.element + " ");
        System.out.println( x:"");
        Display( node:node.right);
    }

}
```

`

```
private TreeNode<E> AddBooking(TreeNode<E> root, Booking booking) {
    if (root == null) {
        root = new TreeNode<E>((E) booking);
        return root;
    }

    if (booking.getBookingNum() < ((Booking) root.element).getBookingNum()) {
        root.left = AddBooking( root:root.left, booking);
    } else if (booking.getBookingNum() > ((Booking) root.element).getBookingNum()) {
        root.right = AddBooking( root:root.right, booking);
    }
    return root;
}

public TreeNode<E> AddBooking(Booking booking) {
    return AddBooking( root:Root, booking);

}
```

```
private Booking searchByBookingNum(TreeNode<E> root, int num) {
    if (root == null) {
        return null;
    }

    if (((Booking) root.element).getBookingNum() == num) {

        return (Booking) root.element;

    }
    if (num < ((Booking) root.element).getBookingNum()) {
        return searchByBookingNum( root:root.left, num);
    } else {
        return searchByBookingNum( root:root.right, num);
    }
}
```

`

```
public TreeNode<E> RoomSearch(TreeNode<E> root, E ele, TreeNode<E> available) {
    if (root == null) {
        return null;
    }
    if (ele == root.element) {
        return root;
    } else {
        if (root.left != null) {
            available = search( root:root.left,  e:ele,  result:available);
        }

        if (root.right != null) {
            available = search( root:root.right,  e:ele,  result:available);
        }

        return available;
    }
}

public Booking searchByBookingNum(int num) {
    return searchByBookingNum( root:Root, num);
}
```

`

III.3 Stack

III.3.1 Method #1 double calculateTotalAmount()

III.3.2 Method #2 Payment isPaymentCompleted(paymentId)

III.3.3 Method #4 void display()

```
// to sum total of amount
public double calculateTotalAmount() {
    double total = 0;
    Node<E> current = top;

    while (current != null) {
        Payment payment = (Payment) current.getElement();
        total += payment.getAmount(); // add new Amount to total
        current = current.getNext();
    }
    return total; // return Total of Amount
}

// Check if payment is Completed
public Payment isPaymentCompleted(int paymentId) {
    Node<E> current = top;
    while (current != null) {

        Payment payment = (Payment) current.getElement();
        if (payment.getPaymentId() == paymentId) {
            return payment; // return if Payment is Completed
        }
        current = current.getNext();
    }
    return null; // return null if Payment is not Completed
}
```

`

IV Main Class

```
Menu:
1. Add Guest
2. Find Guest
3. Remove Guest
4. Print All Guests
5. search Booking
6. cancel Booking
7. Exit
1
Enter Guest ID: 6666
Enter Guest Name: lama
Enter Phone Number: 5654
Guest added successfully.

Menu:
1. Add Guest
2. Find Guest
```

`

V Work distribution

	لجين	ليان	غالية	شهد	رغد
Data Structure#1	☑	☑	☑	☑	☑
Data Structure#1 Method#1	☑	☑	☑	☑	☑
Data Structure#1 Method#2	☑	☑	☑	☑	☑
Data Structure#1 Method#3	☑	☑	☑	☑	☑
Data Structure#2	☑	☑	☑	☑	☑
Data Structure#2 Method#1	☑	☑	☑	☑	☑
Data Structure#2 Method#2	☑	☑	☑	☑	☑
Data Structure#2 Method#3	☑	☑	☑	☑	☑
Data Structure#3	☑	☑	☑	☑	☑
Data Structure#3 Method#1	☑	☑	☑	☑	☑
Data Structure#3 Method#2	☑	☑	☑	☑	☑
Data Structure#3 Method#3	☑	☑	☑	☑	☑

☑ All students worked together